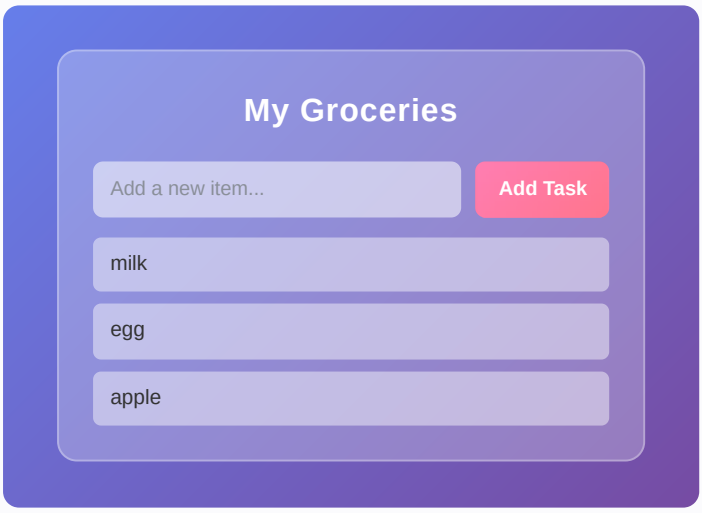
WEEK 01

React 1주차 복습

Todo List로 배우는 React 기초

교수님 수업 코드 `TodoApp.jsx` · `Todoapp.css` 를
한 줄도 바꾸지 않고, “왜 이렇게 동작하는지” 중심으로 다시 읽는 자료



다루는 개념 컴포넌트 · import · useState · onChange/value · Spread · onClick · onKeyDown · index · filter · map · key · CSS 연결

차례

1 이번 주 학습 목표 3

2 TodoApp 전체 구조 4

3 React 컴포넌트와 import 7

4 useState와 상태 관리 9

5 입력창과 onChange 11

6 할 일 추가와 Spread Syntax 13

7 클릭 이벤트와 키보드 이벤트 15

8 배열 index 17

9 filter를 이용한 삭제 17

10 map을 이용한 목록 렌더링 19

11 JSX의 key, onDoubleClick, {todo} 20

12 전체 실행 흐름 21

13 CSS 구조 간단 정리 23

14 핵심 개념 요약 25

15 복습 문제 · 정답과 해설 26

이 자료 읽는 법

표시	의미
POINT	그 절에서 꼭 가져가야 할 핵심
TIP	알아두면 이해가 쉬워지는 보충 설명 (시험 범위 밖일 수 있음)
주의	실제로 공부하면서 헷갈렸던 부분. **헷갈림 주의 1~8**로 번호를 붙였다
L12	TodoApp.jsx의 12번째 줄. 코드 블록 왼쪽 숫자와 같다

POINT 이 자료의 원칙

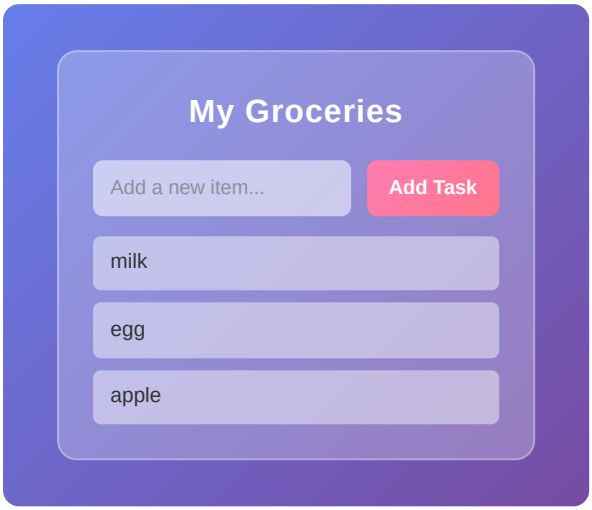
코드는 교수님 원본 그대로다. 더 좋은 방식으로 고치지 않고, 아직 안 배운 개념은 꼭 필요한 곳에만 짧게 TIP으로 표시했다. 목표는 이 PDF만 보고 **TodoApp.jsx 전체 흐름을 내 말로 설명할 수 있게 되는 것**이다.

1

이번 주 학습 목표

이번 주 코드는 짧지만, React 앱이 움직이는 기본 원리가 거의 다 들어 있다.

이 앱이 하는 일



사용자 행동	앱의 반응
입력창에 글자 입력	입력한 글자가 입력창에 보인다
Add Task 클릭 또는 Enter	목록에 항목이 추가되고 입력창이 비워진다
공백만 입력하고 추가	아무 일도 일어나지 않는다
목록 항목 더블클릭	그 항목이 목록에서 사라진다

이 자료를 다 읽고 나면 할 수 있어야 하는 것

- [ ] `TodoList` 가 왜 “함수형 컴포넌트”인지, `return` 이 무엇을 돌려주는지 설명할 수 있다
- [ ] `useState` 가 돌려주는 두 값의 역할과, 초기값 `[]` 와 `''` 를 고른 이유를 말할 수 있다
- [ ] `onChange` 와 `value` 가 각각 무슨 일을 하는지 구분할 수 있다
- [ ] `[...todos, input]` 이 기존 목록을 지키면서 새 배열을 만드는 원리를 설명할 수 있다
- [ ] `onClick` , `onKeyDown` , `onDoubleClick` 이 언제 실행되는지 말할 수 있다
- [ ] `filter` 가 “남길 것을 고르는 함수”라는 관점으로 삭제를 설명할 수 있다
- [ ] `map` 이 배열을 `<li>` 목록으로 바꾸는 과정을 설명할 수 있다
- [ ] milk를 입력하고 추가해서 화면에 나타나기까지의 전체 흐름을 순서대로 말할 수 있다

TIP 공부 순서 추천

처음엔 2장(전체 구조)과 12장(전체 흐름)을 먼저 훑어 큰 그림을 잡고, 3~11장에서 부품을 하나씩 이해한 뒤, 다시 12장으로 돌아와 흐름을 스스로 말해보는 순서가 효과적이다.

2

TodoApp 전체 구조

먼저 원본 코드를 통째로 본다. 앞부분(1–28행)은 “준비와 동작”, 뒷부분(30–60행)은 “화면”이다.

TodoApp.jsx · 1–28행 — 준비(import) · 기억(state) · 동작(함수)

```jsx
import React, { useState } from 'react';
import './css/Todoapp.css';

function TodoList() {
    const [todos, setTodos] = useState([]);
    const [input, setInput] = useState('');

    // 할 일 추가 함수
    const handleAddTask = () => {
        if (input.trim() !== '') {
            setTodos([...todos, input]);
            setInput('');
        }
    };

    // 엔터 키 감지 함수
    const handleKeyDown = (e) => {
        if (e.key === 'Enter') {
            handleAddTask();
        }
    };

    // ✨ 추가된 부분: 할 일 삭제 함수 (더블클릭 시 실행)
    const handleDeleteTask = (indexToDelete) => {
        // filter를 사용하여 클릭한 인덱스와 다른 항목들만 남겨서 새로운 배열 생성
        const newTodos = todos.filter((todo, index) => index !== indexToDelete);
        setTodos(newTodos);
    };
```

행	구역	한 줄 설명
L1–2	① 준비	필요한 기능(React, useState)과 CSS 파일을 가져온다
L5–6	② 기억	앱이 기억할 값 2개: 할 일 목록 `todos`, 입력 중인 글자 `input`
L9–28	③ 동작	추가 `handleAddTask` · Enter 감지 `handleKeyDown` · 삭제 `handleDeleteTask`

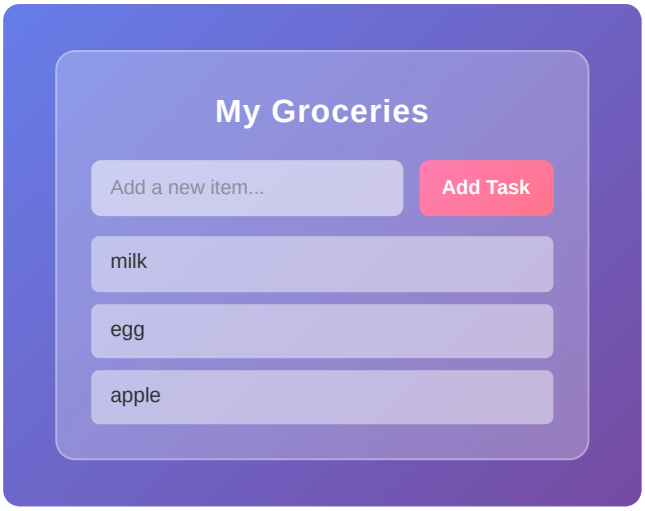
TodoApp.jsx · 29–60행 — 화면(return JSX) · 내보내기(export)

```
29
30      return (
31          <div className="todo-container">
32              <h2>My Groceries</h2>
33
34              <div className="input-group">
35                  <input
36                      value={input}
37                      onChange={(e) => setInput(e.target.value)}
38                      onKeyDown={handleKeyDown}
39                      placeholder="Add a new item..."
40                  />
41                  <button onClick={handleAddTask}>Add Task</button>
42              </div>
43
44              <ul className="todo-list">
45                  {todos.map((todo, index) => (
46                      <li
47                          key={index}
48                          onDoubleClick={() => handleDeleteTask(index)} /* ✨ 추가: 더블클릭 이벤트 연결 */
49                          title="더블클릭하여 삭제" /* 마우스를 올리면 힌트 표시 */
50                          style={{ cursor: 'pointer' }} /* 마우스 커서를 손가락 모양으로 변경 */
51                      >
52                          {todo}
53                      </li>
54                  ))}
55              </ul>
56          </div>
57      );
58  }
59
60  export default TodoList;
```

행	구역	한 줄 설명
L30–57	④ 화면	제목, 입력창+버튼, 목록을 JSX로 그려서 돌려준다
L60	⑤ 내보내기	다른 파일에서 `TodoList`를 가져다 쓸 수 있게 내보낸다

구조 한눈에 보기

```
function TodoList() { … }
  └ ② 기억 todos, input
  └ ③ 동작 추가 · Enter 감지 · 삭제
  └ ④ 화면 return ( JSX )
      └ div.todo-container
          └ h2 · My Groceries
          └ div.input-group → input, button
          └ ul.todo-list → li (todos 개수만큼)
```



h2 제목
input + **button** (div.input-group)
li 반복 (ul.todo-list)

이 앱을 움직이는 단 하나의 원리

사용자 행동 (입력 · 클릭 · 키 · 더블클릭) → **이벤트 함수 실행** (set 함수 호출) → **state 변경** → **React가 다시 그림** (새 state로 화면 갱신)

이 코드에서 화면을 직접 고치는 코드는 한 줄도 없다. 모든 동작은 **state를 바꾸는 것**으로 끝나고, 화면을 새로 그리는 일은 React가 맡는다. 앞으로 나오는 모든 절은 결국 이 고리의 한 칸을 자세히 보는 것이다.

POINT 큰 그림 한 문장

“ `TodoList` 는 `todos` 와 `input` 이라는 두 state를 기억하고, 이벤트가 생기면 set 함수로 state를 바꾸고, React는 바뀐 state로 JSX를 다시 그린다.”

3

React 컴포넌트와 import

어떤 문제를 해결할까?

웹 화면을 HTML 한 파일에 전부 적으면 금방 길어지고, 어디가 무슨 역할인지 알기 어려워진다. React는 화면을 **컴포넌트**(화면의 한 조각을 만드는 함수)로 나눠서, 조각마다 "무엇을 기억하고, 어떻게 동작하고, 어떻게 보이는지"를 한곳에 모아둔다.

> **TIP 비유: 레고 블록**
> 컴포넌트는 레고 블록 설계도다. `TodoList`라는 블록 안에 제목·입력창·목록이 들어 있고, 나중에 다른 화면에서 이 블록을 통째로 끼워 쓸 수 있다.

TodoApp.jsx · 4–6행 (함수 시작 부분)

```
4  function TodoList() {
5      const [todos, setTodos] = useState([]);
6      const [input, setInput] = useState('');
```

함수형 컴포넌트

- `function TodoList() { ... }` — 평범한 자바스크립트 함수 모양이다. 이렇게 **함수로 만든 컴포넌트**를 함수형 컴포넌트라고 부른다.
- 컴포넌트 이름은 `TodoList`처럼 **대문자로 시작**한다. React는 대문자로 시작해야 HTML 태그(`div`, `li`)가 아닌 컴포넌트로 알아본다.

return은 "My Groceries" 글자만 돌려주는 게 아니다

L30–57을 줄여서 본 모양

```
return (
    <div className="todo-container">
        <h2>My Groceries</h2>
        <div className="input-group">···</div>
        <ul className="todo-list">···</ul>
    </div>
);
```

`return ( ··· )` 괄호 안 전체가 하나의 **JSX 덩어리**다. 가장 바깥의 `div.todo-container` 안에 제목, 입력 영역, 목록이 모두 들어 있으므로, `return`은 **이 화면 구조 전체**를 돌려준다.

`h2`의 "My Groceries"는 그 구조 안에 들어 있는 한 부분일 뿐이다.

JSX란?

JSX는 자바스크립트 파일 안에서 HTML처럼 생긴 문법으로 화면을 적는 방법이다. 생긴 건 HTML과 비슷하지만 자바스크립트 안에 있기 때문에 규칙이 조금 다르다.

JSX 규칙	이 코드에서	이유
`{ }` 안에는 자바스크립트 값을 넣는다	`value={input}`, `{todo}`, `{todos.map(...)}`	고정된 글자가 아니라 state 같은 "변하는 값"을 화면에 넣기 위해
`class` 대신 `className`	`className="todo-container"`	`class`는 자바스크립트에서 이미 쓰는 예약어라서
return하는 JSX는 바깥이 하나로 감싸져야 한다	전체를 `div.todo-container` 하나로 감쌈	함수는 값을 하나만 돌려줄 수 있기 때문

import — 필요한 것 가져오기

TodoApp.jsx · 1–2행

```
1  import React, { useState } from 'react';
2  import './css/Todoapp.css';
```

코드	의미
`import React`	React 라이브러리를 가져온다. JSX와 컴포넌트를 쓰는 바탕이 된다.
`{ useState }`	React가 제공하는 여러 기능 중 **useState 하나를 이름으로 골라서** 가져온다. 중괄호는 "이 이름의 기능만 꺼내겠다"는 뜻이다.
`from 'react'`	설치된 react 패키지에서 가져온다는 뜻이다.
`import './css/Todoapp.css'`	이 컴포넌트가 쓸 **CSS 파일을 연결**한다. 변수로 받는 게 아니라 "이 스타일을 적용해줘"라는 의미라서 `from`이 없다.

src/ (예상 구조)

```
└ TodoApp.jsx
└ css/
    └ Todoapp.css
```

`./`는 "지금 이 파일(TodoApp.jsx)이 있는 폴더"를 뜻한다. 그래서 `./css/Todoapp.css`는 같은 폴더 안 `css` 폴더의 파일이다. 파일 이름의 대소문자까지 정확히 맞아야 한다.

TIP 최근 React 환경 참고

최근 설정에서는 `import React`를 적지 않아도 JSX가 동작하는 경우가 많다. 하지만 적어도 문제는 없고, 교수님 코드는 적는 방식이다. 지금은 그대로 따라가면 된다.

export — 다른 파일에서 쓸 수 있게 내보내기

TodoApp.jsx · 60–60행

```
60  export default TodoList;
```

`export default TodoList;`는 "이 파일의 대표 컴포넌트는 `TodoList`다"라고 내보내는 문장이다. 보통 `App.jsx` 같은 다른 파일에서 이 컴포넌트를 가져와 화면에 올린다. 내보내지 않으면 다른 파일에서 쓸 수 없다.

POINT 파일 이름과 컴포넌트 이름이 다르다

파일은 `TodoApp.jsx`인데 함수 이름은 `TodoList`다. 둘이 달라도 동작에는 문제가 없다. 코드를 읽을 때 "파일 이름 = 컴포넌트 이름"이라고 가정하지 말고 **export 줄을 확인**하는 습관을 들이자.

4

useState와 상태 관리

어떤 문제를 해결할까?

할 일 앱은 “지금 목록에 뭐가 있는지”, “입력창에 뭐가 적혀 있는지”를 **기억**해야 하고, 그 값이 바뀌면 **화면도 바뀌어야** 한다. 함수 안의 평범한 변수는 값을 바꿔도 React가 알아채지 못해서 화면이 다시 그려지지 않는다. 그래서 React에게 “이 값은 지켜보고 있다가, 바뀌면 화면을 다시 그려줘”라고 맡기는 도구가 **useState**다.

이렇게 컴포넌트가 기억하고, 바뀌면 화면에 반영되는 값을 **state(상태)**라고 부른다.

> **TIP** **비유: 게시판과 담당자**
>
> state는 교실 게시판에 적힌 내용이다. 게시판 내용은 누구나 **읽을 수** 있지만, **고치는 건 담당자(set 함수)만** 한다.
> 담당자가 고치면 React가 “게시판이 바뀌었네” 하고 화면을 새로 그린다.

기본 구조

const [현재 상태 , 상태를 변경하는 함수] = useState(초기값);

TodoApp.jsx · 5–6행

```
5  const [todos, setTodos] = useState([]);
6  const [input, setInput] = useState('');
```

이름	역할	예시 값
`todos`	현재 **할 일 목록**을 저장하는 state	`["milk", "egg"]`
`setTodos`	`todos` state를 **변경하는 함수**	`setTodos([...])`
`[]`	`todos`의 초기값 — **빈 배열**	처음엔 목록이 비어 있음
`input`	현재 **입력창의 문자열**을 저장하는 state	`"milk"`
`setInput`	`input` state를 **변경하는 함수**	`setInput('')`
`''`	`input`의 초기값 — **빈 문자열**	처음엔 입력창이 비어 있음

> **POINT** **대괄호 [ ] 의 의미**
>
> `useState([])`는 두 개짜리 묶음을 돌려주고, `const [todos, setTodos]`는 그 묶음을 **순서대로 꺼내 이름을 붙이는** 문법이다. 첫 번째가 현재 값, 두 번째가 변경 함수라는 **순서**가 중요하다. 이름은 `todos` / `setTodos` 처럼 “값 / set+값” 짝으로 짓는 것이 관례다.

[] 와 '' 의 차이 — 무엇을 담느냐가 결정한다

todos → 배열 `[]`

`"milk"` `"egg"` `"apple"`

할 일은 **여러 개**다. 여러 값을 순서대로 담는 상자인 **배열**이 필요하다.

```
["milk", "egg", "apple"]
```

처음엔 아무것도 없으니 빈 배열 `[]`로 시작한다.

input → 문자열 `''`

`"milk"`

입력창에 적힌 내용은 **지금 입력 중인 글자 한 줄**이다. 그래서 **문자열** 하나면 된다.

```
"milk"
```

처음엔 입력창이 비어 있으니 빈 문자열 `''`로 시작한다.

	todos	input
초기값	`[]` 빈 배열	`''` 빈 문자열
담는 것	할 일 **여러 개**	입력 중인 **글자 하나(한 줄)**
값 예시	`["milk", "egg"]`	`"milk"`
쓰이는 곳	`todos.map(...)`, `todos.filter(...)` — 배열 기능	`value={input}`, `input.trim()` — 문자열 기능

주의 헷갈림 주의 1 — 초기값의 모양

`todos`의 초기값 `[]` = **빈 배열**, `input`의 초기값 `''` = **빈 문자열**이다. 둘 다 "비어 있다"는 점은 같지만 **자료의 종류가 다르다**.

초기값의 종류가 틀리면 코드가 깨진다. 예를 들어 `todos`를 `useState('')`로 시작하면, 문자열에는 `map` 기능이 없어서 `todos.map(...)` 부분에서 오류가 난다. 초기값은 "이 state에 앞으로 어떤 종류의 값이 들어올지"를 정하는 약속이다.

생각해보기

`todos`를 배열이 아니라 문자열 `"milk egg apple"` 하나로 저장했다면, egg 하나만 지우는 일은 쉬웠을까? 배열을 쓰면 **index**(8장)로 항목 하나하나를 구분할 수 있다는 점이 핵심이다.

5

입력창과 onChange

어떤 문제를 해결할까?

사용자가 입력창에 적은 글자를 앱이 알아야 나중에 목록에 추가할 수 있다. 그래서 **입력창 내용을 state(`input`)와 항상 똑같이 맞춰두는** 구조를 만든다. 입력창과 state를 이렇게 묶는 방식을 **제어 컴포넌트**(state가 입력창을 제어하는 방식)라고 부른다.

TodoApp.jsx · 35–40행

```jsx
<input
    value={input}
    onChange={(e) => setInput(e.target.value)}
    onKeyDown={handleKeyDown}
    placeholder="Add a new item..."
/>
```

입력 한 번의 흐름

사용자가 milk 입력 — 키보드로 글자를 친다

↓

onChange 발생 — 입력창 내용이 바뀌었다는 신호

↓

e.target.value = "milk" — 지금 입력창에 적힌 글자

↓

setInput("milk") — state 변경 요청

↓

input state가 "milk"로 변경 — React가 다시 그림

↓

value={input} — 입력창에 state 값을 넣음

↓

입력창에 milk 표시

e, e.target, value

표현	뜻
`e`	**이벤트 객체**. "무슨 일이, 어디서 일어났는지" 정보를 담은 꾸러미. React가 함수에 넣어준다.
`e.target`	이벤트가 일어난 **그 요소** — 여기서는 `<input>`
`e.target.value`	그 입력창에 **지금 적혀 있는 글자**

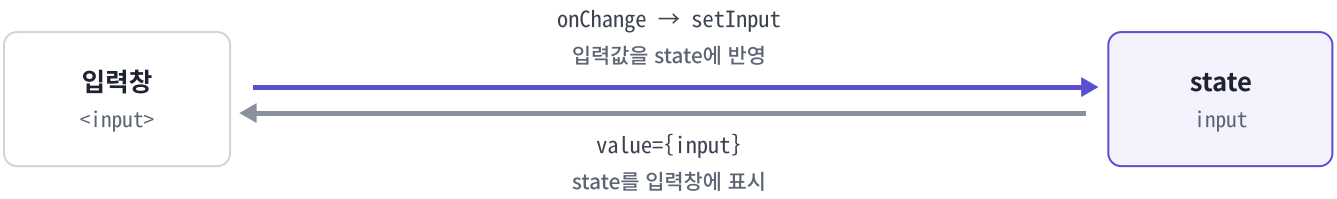
POINT **onChange는 글자 하나마다 실행된다**

milk를 입력하면 onChange는 한 번이 아니라 **네 번** 실행된다.

```
"m" → setInput("m")
"mi" → setInput("mi")
"mil" → setInput("mil")
"milk" → setInput("milk")
```

왼쪽 흐름도는 마지막 한 번을 대표로 그린 것이다.

onChange와 value — 역할이 반대 방향이다



	onChange={(e) => setInput(e.target.value)}	value={input}
방향	입력창 → **state**	state → **입력창**
하는 일	사용자가 입력한 값을 **state에 반영**	현재 state 값을 **입력창에 표시**
언제	입력창 내용이 바뀔 때마다 실행	화면을 그릴 때마다 적용
비유	받아쓴 내용을 **원본 노트에 적기**	원본 노트를 **보여주는 화면**

주의 헷갈림 주의 3 — value는 저장하지 않는다

`value={input}`이 입력값을 state에 저장하는 것이 **아니다**.

state를 바꾸는 건 **onChange → setInput**이고, `value={input}`은 **현재 state를 입력창에 보여줄 뿐**이다. 저장(onChange)과 표시(value)는 서로 다른 줄이 맡는다.

왜 굳이 두 개가 필요할까?

`value={input}`이 있으면 입력창에는 **항상 state 값만** 보인다. 그래서 onChange가 state를 바꿔주지 않으면, 키보드를 눌러도 입력창 글자가 바뀌지 않는다. 반대로 이 구조 덕분에 `setInput('')`만 호출해도 입력창이 비워진다(6장). 입력창을 코드로 비울 수 있는 이유가 바로 `value={input}`이다.

TIP 실험해보기

수업 코드를 복사한 연습 파일에서 `onChange` 줄만 잠깐 지우고 입력해보면, 입력창에 글자가 들어가지 않는 것을 직접 확인할 수 있다. 확인 후에는 원래대로 되돌린다.

6

할 일 추가와 Spread Syntax

TodoApp.jsx · 8–14행

```
8  // 할 일 추가 함수
9  const handleAddTask = () => {
10     if (input.trim() !== '') {
11         setTodos([...todos, input]);
12         setInput('');
13     }
14 };
```

코드	의미
`if (input.trim() !== '')`	앞뒤 공백을 지운 결과가 빈 문자열이 **아닐 때만** 안쪽을 실행한다. 빈 칸이나 공백만 입력하면 추가하지 않는다.
`...todos`	기존 `todos` 배열의 내용을 **펼쳐서** 가져온다.
`[...todos, input]`	기존 항목을 유지하고 끝에 `input`을 붙인 **새로운 배열**을 만든다.
`setTodos(...)`	그 새 배열로 **todos state를 변경**한다.
`setInput('')`	추가한 뒤 입력창을 **다시 비운다**.

trim()이 필요한 이유

`trim()`은 문자열의 **앞과 뒤 공백**을 잘라낸 결과를 돌려준다. 공백만 입력한 경우도 "빈 입력"으로 걸러내기 위해 쓴다.

input 값	input.trim()	!== '' ?	결과
`"milk"`	`"milk"`	true	추가됨
`""`	`""`	false	추가 안 됨
`"  "` (공백 3개)	`""`	false	추가 안 됨
`" egg "`	`"egg"`	true	추가됨 — 단, 저장되는 값은 `" egg "`

POINT **검사에만 trim을 쓴다**

11행은 `input.trim()`이 아니라 `input`을 넣는다. 그래서 공백 검사는 하지만, 저장할 때는 **입력한 그대로**(앞뒤 공백 포함) 저장된다. 코드를 "대충 이런 뜻"으로 읽지 않고 한 글자씩 확인해야 알 수 있는 부분이다.

예제로 따라가기

실행 전

```
todos = ["milk"]
input = "egg"
```

→

실행

```
setTodos([...todos, input])
setInput('')
```

→

실행 후

```
todos = ["milk", "egg"]
input = ""
```

Spread Syntax (...) — 상자에서 꺼내 펼치기

Spread Syntax(전개 문법)는 `...` 세 개의 점으로, 배열 안의 항목들을 **하나씩 꺼내 그 자리에 펼쳐놓는** 문법이다.

TIP 비유: 장바구니 옮겨 담기

`[...todos, input]`은 새 장바구니를 하나 가져와서, 기존 장바구니(`todos`)의 물건을 **전부 꺼내 옮겨 담고**, 마지막에 새 물건(`input`)을 넣는 것이다. 기존 물건은 그대로 다 들어 있다.

✔ 교수님 코드: `[...todos, input]`

```
todos = ["milk", "egg"]
input = "banana"

[...todos, input]
// → ["milk", "egg", "banana"]
```

기존 항목 + 새 항목. 목록이 쌓인다.

✘ 만약 `[input]`으로 썼다면

```
todos = ["milk", "egg"]
input = "banana"

[input]
// → ["banana"]
```

새 항목만 있는 배열. `setTodos([input])`이면 milk, egg가 **사라진다**.

쓴 코드	만들어지는 배열	펼친 뒤 모습
`[...todos, input]`	`["milk", "egg", "banana"]`	`["milk", "egg", "banana"]` ← `...todos`가 milk, egg로 펼쳐짐
`[todos, input]`	`[["milk", "egg"], "banana"]`	점이 없으면 배열이 **통째로 한 칸**에 들어가 버린다
`[input]`	`["banana"]`	기존 목록 없음

주의 헷갈림 주의 2 — ...todos가 state를 바꾸는 게 아니다

`...todos`는 **새 배열을 만드는 재료**로 쓰일 뿐, 그 자체로 `todos`를 바꾸지 않는다. 실제 state 변경은 **`setTodos`**가 한다.

주의 헷갈림 주의 8 — 추가할 때 기존 todos는 사라지지 않는다

새 값을 추가해도 기존 항목이 지워지는 것이 아니다. `[...todos, input]`으로 **기존 값 + 새로운 값** 형태의 새 배열이 만들어지고, 그 배열이 새 `todos`가 된다.

TIP 왜 todos.push(input)가 아니라 새 배열일까? (지금은 이 정도만)

React는 set 함수에 **새 배열**이 들어와야 "값이 바뀌었다"고 확실히 알아채고 다시 그린다. 기존 배열을 직접 고치는 대신 새 배열을 만드는 방식을 React에서 기본으로 쓴다는 것만 기억해두자.

7

클릭 이벤트와 키보드 이벤트

이벤트는 클릭, 키 입력, 더블클릭처럼 **사용자가 한 행동**이다. JSX에서는 `on + 행동이름` 속성에 "그 행동이 일어나면 실행할 함수"를 연결한다.

속성	이 코드에서 연결된 함수	언제 실행?
`onChange`	`(e) => setInput(e.target.value)`	입력창 내용이 바뀔 때
`onClick`	`handleAddTask`	버튼을 클릭할 때
`onKeyDown`	`handleKeyDown`	입력창에서 **아무 키나** 누를 때
`onDoubleClick`	`() => handleDeleteTask(index)`	목록 항목을 더블클릭할 때

버튼 클릭 — onClick

TodoApp.jsx · 41–41행

```
41  <button onClick={handleAddTask}>Add Task</button>
```

Add Task 버튼 클릭 → **onClick 발생** → **handleAddTask 실행**
6장의 추가 과정

onClick={handleAddTask} 와 onClick={handleAddTask()} 의 차이

	`onClick={handleAddTask}`	`onClick={handleAddTask()}`
뜻	함수 **자체를 넘긴다**. "클릭하면 이걸 실행해줘"	함수를 **지금 바로 실행**하고, 그 결과를 넘긴다
실행 시점	**클릭할 때**	**화면을 그리는 순간** (클릭과 무관)
결과	의도대로 동작	handleAddTask는 아무것도 return하지 않으므로, 클릭해도 원하는 동작이 일어나지 않는다

TIP **비유: 전화번호 vs 지금 전화 걸기**

`handleAddTask` 는 "불이 나면 이 번호로 전화해"라고 **번호를 적어 두는 것**이고, `handleAddTask()` 는 적는 순간 **바로 전화를 거는 것**이다. 이벤트에는 번호(함수 자체)를 적어 둔다.

POINT **그럼 onDoubleClick은 왜 () => 로 감쌌을까?**

삭제할 때는 "몇 번째 항목인지" `index` 를 **함께 넘겨야** 한다. 그런데 `onDoubleClick={handleDeleteTask(index)}` 라고 쓰면 괄호 때문에 그리는 순간 바로 실행된다. 그래서 `() => handleDeleteTask(index)` 처럼 **"더블클릭하면 이걸 실행해줘"라는 새 함수로 한 번 감싸서** 넘긴다. (11장에서 다시 본다)

키보드 이벤트 — onKeyDown

TodoApp.jsx · 17–21행

```
17 const handleKeyDown = (e) => {
18     if (e.key === 'Enter') {
19         handleAddTask();
20     }
21 };
```

TodoApp.jsx · 38행 (input 안)

```
38 onKeyDown={handleKeyDown}
```

`onKeyDown={handleKeyDown}` 때문에 입력창에서 **키를 누를 때마다** `handleKeyDown`이 실행된다. 함수 안에서 `e.key`로 **어떤 키가 눌렸는지** 확인하고, Enter일 때만 `handleAddTask()`를 부른다. 여기서는 이벤트 속성이 아니라 **함수 안에서 직접 실행**하는 것이므로 괄호 `()`를 붙이는 게 맞다.

누른 키	handleKeyDown 실행?	e.key 값	e.key === 'Enter'	결과
A	**실행됨**	`'a'`	false	아무 일 없음 (글자는 입력됨)
Space	**실행됨**	`' '`	false	아무 일 없음
Enter	**실행됨**	`'Enter'`	true	`handleAddTask()` 실행

주의 헷갈림 주의 5 — Enter를 눌러야 실행되는 함수가 아니다

"Enter를 눌러야 handleKeyDown이 실행된다"가 **아니다**. A, Space, Enter 등 **어떤 키를 눌러도 handleKeyDown은 실행**되고, 그 안의 `if (e.key === 'Enter')`가 Enter인지 **검사**해서 Enter일 때만 `handleAddTask`를 실행한다. "문지기는 모든 사람을 확인하지만, 통과시키는 건 Enter뿐"이라고 생각하면 된다.

주의 헷갈림 주의 4 — 입력창을 비우는 건 Enter가 아니다

Enter를 눌렀기 **때문에** input이 빈 문자열이 되는 것이 아니다. Enter(또는 버튼 클릭)는 `handleAddTask`를 **실행시킬 뿐**이고, 입력창을 비우는 건 그 안의 **`setInput('')`** 이다.

Enter 또는 클릭 → handleAddTask 실행 → (공백 검사 통과 시) setTodos → **setInput('')** → input = ""

그래서 공백만 입력하고 Enter를 누르면 if 안쪽이 실행되지 않아 **입력창도 비워지지 않는다**.

TIP 한글로 테스트할 때 알아둘 것

한글은 자음·모음을 조합하며 입력되기 때문에, 브라우저에 따라 한글 입력 직후 Enter를 누르면 마지막 글자가 **한 번 더 추가되는** 현상이 생길 수 있다. 수업 코드의 논리가 틀린 것이 아니라 한글 조합 입력 방식 때문이다. 흐름을 확인할 때는 영어(milk, egg)로 테스트하면 헷갈리지 않는다.

8

배열 index

index(인덱스)는 배열 안에서 항목의 **위치 번호**다. 자바스크립트 배열의 index는 **0부터 시작**한다.



항목	몇 번째?	index
milk	1번째	**0**
egg	2번째	**1**
apple	3번째	**2**

항목이 3개면 마지막 index는 3이 아니라 **2**다. 규칙: **마지막 index = 개수 − 1**

"몇 번째"는 1부터 세고, index는 **출발점에서 몇 칸 떨어졌는지**라고 생각하면 0부터인 이유가 자연스럽다. 첫 항목은 출발점에서 0칸 떨어져 있다.

> **주의** **헷갈림 주의 7 — index는 0부터**
> 배열의 첫 번째 요소의 index는 **0**이다. 이 코드에서 index는 삭제할 항목을 가리키는 번호로 쓰이므로(9장), 하나만 어긋나도 **엉뚱한 항목이 지워진다**.

9

filter를 이용한 삭제

filter는 "삭제 함수"가 아니다

`filter()`는 배열의 각 요소에 조건을 물어보고, **조건이 true인 요소만 남겨 새로운 배열을 만드는 함수**다. 원래 배열은 건드리지 않는다.

> **TIP** **비유: 체로 거르기**
> 체에 재료를 붓고, 구멍을 **통과한(true) 것만 새 그릇**에 모은다. "egg가 아닌 것만 통과"라는 체를 쓰면, 결과적으로 egg가 빠진 그릇이 생긴다. 삭제는 그 **결과**다.

TodoApp.jsx · 23–28행

```
23 // ✨ 추가된 부분: 할 일 삭제 함수 (더블클릭 시 실행)
24 const handleDeleteTask = (indexToDelete) => {
25     // filter를 사용하여 클릭한 인덱스와 다른 항목들만 남겨서 새로운 배열 생성
26     const newTodos = todos.filter((todo, index) => index !== indexToDelete);
27     setTodos(newTodos);
28 };
```

코드	의미
`(indexToDelete)`	지울 항목의 index를 받는다. 더블클릭한 항목의 index가 들어온다.
`todos.filter(...)`	todos의 요소를 **처음부터 하나씩** 꺼내 조건 함수에 넣어본다.
`(todo, index) =>`	filter가 넣어주는 값: `todo`는 **현재 요소 값**, `index`는 **현재 요소 위치**. 이 코드는 `todo`를 쓰지 않지만, **두 번째 자리의 index를 받으려면 첫 번째 자리도 적어야** 한다.
`index !== indexToDelete`	지울 위치와 **다르면 true → 남김**, 같으면 false → 제외
`setTodos(newTodos)`	만들어진 새 배열로 state를 변경한다.

egg를 지울 때 filter 안에서 일어나는 일

`todos = ["milk", "egg", "apple"]`, egg의 index는 1 → `handleDeleteTask(1)`

차례	todo	index	조건 `index !== 1`	결과	newTodos에
1	`"milk"`	0	`0 !== 1` → **true**	유지	`["milk"]`
2	`"egg"`	1	`1 !== 1` → **false**	제외	`["milk"]`
3	`"apple"`	2	`2 !== 1` → **true**	유지	`["milk", "apple"]`

filter 결과

```
newTodos = ["milk", "apple"]
```

→ **state 변경**

```
setTodos(newTodos)
```

→ **결과**

```
todos = ["milk", "apple"]
```

주의 **헷갈림 주의 6 — filter 자체는 삭제 명령이 아니다**

`filter()`에 "egg를 지워라"라는 명령은 없다. **조건이 true인 요소들만 남긴 새 배열**을 만들고, 그 배열을 `setTodos`로 넣어서 **삭제 효과**를 만드는 것이다. 그래서 `filter`만 하고 `setTodos`를 안 하면 화면에서 아무것도 지워지지 않는다.

POINT **조건을 반대로 쓰면?**

`index === indexToDelete`라고 쓰면 true인 요소가 egg 하나뿐이라, 결과는 `["egg"]` — **지우려던 것만 남는다**. filter는 "무엇을 남길지"를 쓰는 함수라는 걸 보여주는 예다.

10

map을 이용한 목록 렌더링

어떤 문제를 해결할까?

`todos`에는 할 일이 몇 개 들어 있을지 모른다. 0개일 수도, 10개일 수도 있다. `<li>`를 손으로 몇 개 적어둘 수 없으니, **배열의 개수만큼 자동으로 `<li>`를 만들어야** 한다. 이때 쓰는 것이 `map()`이다.

`map()`은 배열의 **각 요소를 하나씩 처리해서 다른 형태로 바꾼 새 배열**을 만든다. filter가 "남길지 말지"를 고른다면, map은 "**모든 요소를 무엇으로 바꿀지**"를 정한다.

> **TIP** **비유: 포장 공장**
>
> 컨베이어 벨트에 재료(milk, egg, apple)가 하나씩 들어오고, 기계가 각각을 **같은 방식으로 포장**(`<li>`)해서 내보낸다. 들어간 개수와 나온 개수가 **같다**.

```
"milk"    →  <li>milk</li>     todo = "milk", index = 0
"egg"     →  <li>egg</li>      todo = "egg", index = 1
"apple"   →  <li>apple</li>    todo = "apple", index = 2
```

TodoApp.jsx · 44–55행

```jsx
<ul className="todo-list">
    {todos.map((todo, index) => (
        <li
            key={index}
            onDoubleClick={() => handleDeleteTask(index)} /* ✨ 추가: 더블클릭 이벤트 연결 */
            title="더블클릭하여 삭제" /* 마우스를 올리면 힌트 표시 */
            style={{ cursor: 'pointer' }} /* 마우스 커서를 손가락 모양으로 변경 */
        >
            {todo}
        </li>
    ))}
</ul>
```

코드	의미
`{todos.map(...)}`	JSX 안에서 자바스크립트를 실행하려고 `{ }`로 감쌌다. map이 만든 `<li>` 배열을 React가 목록으로 그린다.
`(todo, index)`	`todo` = **현재 처리하고 있는 실제 값** (예: `"egg"`) / `index` = **현재 요소의 배열 위치** (예: `1`)
`=> ( <li>···</li> )`	화살표 뒤의 괄호 `( )`는 "이 JSX를 바로 돌려준다"는 뜻이다. 요소마다 `<li>` 하나가 만들어진다.

> **POINT** **다시 그릴 때마다 map이 새로 실행된다**
>
> todos가 `["milk"]`에서 `["milk", "egg"]`로 바뀌면 React가 컴포넌트를 다시 그리고, 그때 `todos.map`도 **다시 실행**되어 `<li>`가 2개 만들어진다. 목록에 "추가하는 코드"가 따로 없는데도 화면에 항목이 늘어나는 이유다.

11

JSX의 key, onDoubleClick, {todo}

TodoApp.jsx · 46–53행

```jsx
<li
    key={index}
    onDoubleClick={() => handleDeleteTask(index)} /* ✨ 추가: 더블클릭 이벤트 연결 */
    title="더블클릭하여 삭제" /* 마우스를 올리면 힌트 표시 */
    style={{ cursor: 'pointer' }} /* 마우스 커서를 손가락 모양으로 변경 */
>
    {todo}
</li>
```

속성 / 표현	역할
`{todo}`	현재 todo 값을 **화면에 글자로 출력**한다. `{ }`가 없으면 변수 값이 아니라 "todo"라는 글자 자체가 나온다.
`key={index}`	React가 **목록의 각 항목을 구분**하기 위해 쓰는 값. 다시 그릴 때 "어떤 li가 어떤 li인지" 알아보는 이름표다. map으로 만든 목록에는 key를 붙인다.
`onDoubleClick={() => handleDeleteTask(index)}`	**더블클릭 이벤트**. 더블클릭하면 그 항목의 index를 넣어 삭제 함수를 실행한다.
`title="더블클릭하여 삭제"`	마우스를 올리면 뜨는 힌트 문구 (HTML 기본 속성)
`style={{ cursor: 'pointer' }}`	마우스 커서를 손가락 모양으로. 바깥 `{ }`는 "JSX에 JS 값 넣기", 안쪽 `{ }`는 **스타일 정보를 담은 객체**다.

TIP **key={index}에 대해 앞으로 배울 것**

이 코드는 index를 key로 쓴다. 지금 앱에서는 문제없이 동작한다. 나중에 더 복잡한 목록에서는 **항목마다 고유한 id**를 key로 쓰는 방식을 배우게 된다. 지금은 "key는 목록 항목의 이름표"까지만 알면 된다.

apple(index 2)을 더블클릭하면

apple 더블클릭 — `todos = ["milk", "egg", "apple"]`

↓

onDoubleClick 발생 — apple을 만들 때 index는 2였다

↓

handleDeleteTask(2)

↓

filter: index 2를 제외한 새 배열 — `["milk", "egg"]`

↓

setTodos(newTodos) — todos state 변경

↓

React 재렌더링 → map — `<li>`가 2개만 만들어짐

↓

화면에서 apple 삭제

POINT **index가 어떻게 "기억"될까?**

map이 apple의 `<li>`를 만들 때, 그 순간의 `index` (2)를 넣어서 `() => handleDeleteTask(index)` 함수를 만든다. 그래서 각 `<li>`는 **자기 index를 품은 함수**를 하나씩 갖고 있다.

milk의 li → `handleDeleteTask(0)`
egg의 li → `handleDeleteTask(1)`
apple의 li → `handleDeleteTask(2)`

이게 7장에서 본 "`() =>`로 감싸는 이유"의 실제 모습이다.

12

전체 실행 흐름

사용자가 milk를 입력하고 Add Task를 누르는 전체 과정. 오른쪽 작은 글씨는 해당 코드 줄이다.

A 입력하기

사용자 입력 — milk 타이핑

↓

onChange — L37

↓

e.target.value — "milk"

↓

setInput — L37

↓

input state 변경 — "milk"

↓

value={input} — 입력창에 milk 표시 · L36

B 추가하기

Add Task 클릭

↓

onClick — L41

↓

handleAddTask — L9

↓

빈 문자열인지 검사 — `input.trim() !== ''` · L10

↓

setTodos([...todos, input]) — L11

↓

setInput('') — L12

C 다시 그리기

todos · input state 변경
todos = ["milk"], input = ""

→ **React 재렌더링**
TodoList 함수가 다시 실행

→ **todos.map()**
L45

→ ** 생성**
L46–53

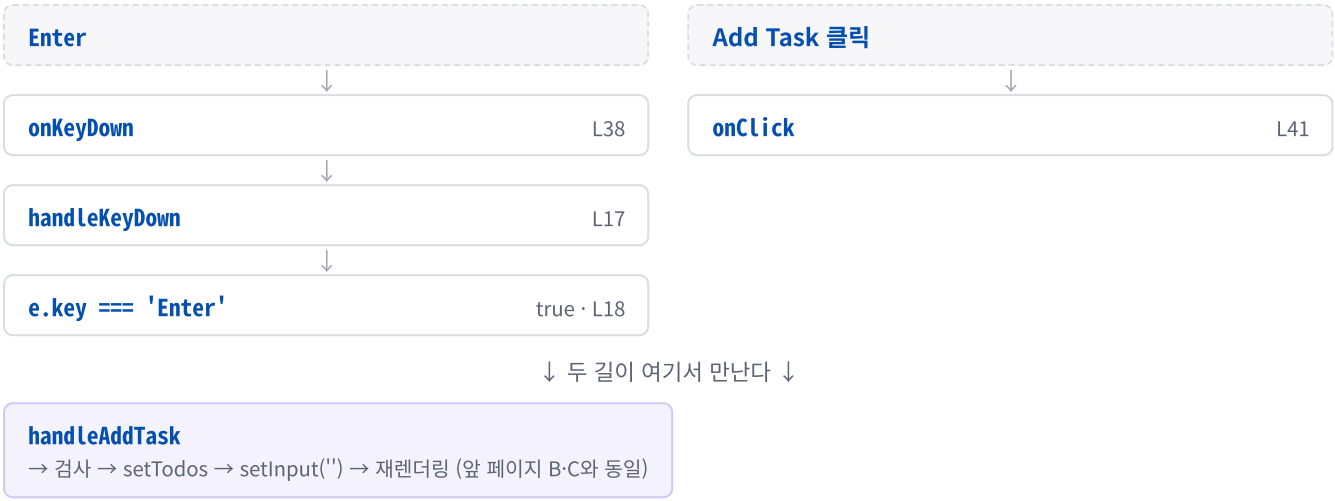
→ **화면에 milk 출력**
입력창도 비어 있음

POINT **setInput('')은 언제 실행될까?**

`setInput('')` 은 화면에 milk가 나온 **다음에** 따로 실행되는 게 아니라, handleAddTask 안에서 `setTodos` **바로 다음 줄**(L12)에서 호출된다. React는 한 함수 안의 state 변경들을 모아서 **한 번에 다시 그리므로**, 화면에는 "milk 추가"와 "입력창 비우기"가 **동시에** 반영된다. 흐름을 설명할 때 "setTodos → setInput('') → 재렌더링" 순서로 말하는 것이 코드와 정확히 일치한다.

Enter로 추가하는 경우

버튼 대신 Enter를 누르면 앞부분만 다르고, **handleAddTask에서 같은 길로 합류**한다.



state 변화 타임라인

시간 순서대로 state와 화면이 어떻게 바뀌는지 표로 보면, “state가 바뀌면 화면이 따라간다”는 원리가 한눈에 보인다.

#	사용자 행동	input	todos	화면
0	(처음 열었을 때)	`''`	`[]`	빈 입력창, 목록 없음
1	milk 입력	`"milk"`	`[]`	입력창에 milk
2	Add Task 클릭	`''`	`["milk"]`	목록에 milk, 입력창 빔
3	egg 입력 후 Enter	`''`	`["milk", "egg"]`	목록에 milk, egg
4	공백 3칸 입력 후 Enter	`"  "`	`["milk", "egg"]`	변화 없음, 공백이 입력창에 남음
5	공백 지우고 apple 추가	`''`	`["milk", "egg", "apple"]`	목록 3개
6	egg 더블클릭	`''`	`["milk", "apple"]`	egg 사라짐

TIP 스스로 설명해보기

이 표의 2번 행을 코드 줄 번호(L37 → L41 → L9 → L10 → L11 → L12 → L45)를 짚어가며 소리 내어 설명해보자. 막힘없이 되면 이번 주 핵심은 다 이해한 것이다.

13

CSS 구조 간단 정리

CSS는 속성을 외우기보다 “어느 부분의 디자인을 담당하는가” 중심으로 본다.

className과 CSS 클래스의 연결

TodoApp.jsx (JSX)

```jsx
<div className="todo-container">
```

Todoapp.css

```css
.todo-container {
    ...
}
```

JSX에서 `className="todo-container"`라고 이름표를 붙이면, CSS에서 **점(.)으로 시작하는** `.todo-container { }` 규칙이 그 요소에 적용된다. 이름이 한 글자라도 다르면 연결되지 않는다.

CSS 선택자	가리키는 JSX 요소
`body`	페이지 전체 바탕 (JSX 밖, HTML 문서의 body)
`.todo-container`	`<div className="todo-container">` — 카드 전체
`.todo-container h2`	todo-container **안에 있는** `<h2>` (띄어쓰기 = “~안의”)
`.input-group`	`<div className="input-group">` — 입력창+버튼 줄
`.todo-container input` / `button`	컨테이너 안의 `<input>`, `<button>` — className 없이 태그 이름으로 지정
`.todo-list` / `.todo-list li`	`<ul className="todo-list">`와 그 안의 `<li>`들

화면 영역 지도

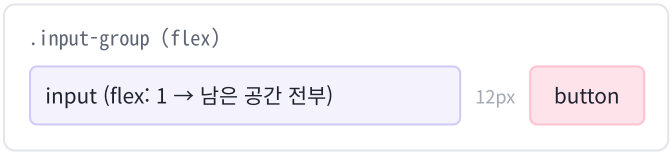


영역별 역할

선택자	담당하는 디자인	눈여겨볼 속성
`body`	화면 전체 배경과 카드의 **가운데 정렬**	`display: flex` + `justify-content/align-items: center`, `linear-gradient`
`.todo-container`	반투명 **유리 느낌 카드** (글래스모피즘)	`background: rgba(...)`, `backdrop-filter: blur`, `border-radius`, `max-width: 400px`
`.todo-container h2`	제목 크기와 아래 여백	`font-size`, `margin-bottom`
`.input-group`	입력창과 버튼을 **가로 한 줄**로 배치	`display: flex`, `gap: 12px`
`input`	입력창 모양, 남은 폭 채우기	`flex: 1`, `:focus` 시 배경이 더 밝아짐
`button`	분홍 그라데이션 버튼	`cursor: pointer`, `:hover`, `:active`
`.todo-list`	목록 전체: 점 없애고 **세로로 간격** 두기	`list-style-type: none`, `flex-direction: column`, `gap`
`.todo-list li`	할 일 한 줄 한 줄의 **카드 모양**	반투명 배경, `padding`, `border-radius`, `text-align: left`

display: flex — 줄 세우기

`display: flex`를 준 요소는 **안에 든 자식들을 한 줄로 나란히** 세운다. 기본 방향은 가로다.



`.todo-list`는 `flex-direction: column`이라 **세로**로 줄 세우고 `gap`으로 항목 사이를 띄운다.

:hover와 :active — 상태별 디자인

선택자	언제	효과
`button`	평소	기본 모양
`button:hover`	마우스를 **올렸을 때**	2px 위로 뜨고 그림자 진해짐
`button:active`	**누르고 있는 동안**	1px 아래로 눌림
`input:focus`	입력창을 **클릭해 입력 중**	배경이 더 밝아짐

`transition`이 있어서 이 변화가 뚝 끊기지 않고 **부드럽게** 바뀐다. 버튼이 "눌리는 느낌"을 주는 작은 사용자 경험 장치다.

POINT **React와 CSS의 역할 분담**

React(JSX)는 **무엇을, 몇 개 보여줄지**(구조와 데이터)를 정하고, CSS는 **어떻게 보일지**(모양)를 정한다. li가 몇 개가 되든 `.todo-list li` 규칙 하나로 전부 같은 모양이 되는 이유다.

14

핵심 개념 요약

코드	한 줄 의미
`function TodoList() { return (...) }`	화면 구조 전체를 JSX로 돌려주는 **함수형 컴포넌트**
`import React, { useState } from 'react'`	React와 useState 기능을 가져오기
`import './css/Todoapp.css'`	CSS 파일을 연결해 스타일 적용
`const [todos, setTodos] = useState([])`	[현재 상태, 변경 함수] = useState(초기값) · 목록은 **배열**
`const [input, setInput] = useState('')`	입력 중인 글자는 **문자열**
`onChange={(e) => setInput(e.target.value)}`	입력창 → state (**반영**)
`value={input}`	state → 입력창 (**표시**)
`input.trim() !== ''`	공백뿐인 입력은 추가하지 않기
`setTodos([...todos, input])`	기존 항목 + 새 항목으로 **새 배열** 만들어 state 변경
`setInput('')`	추가 후 입력창 비우기
`onClick={handleAddTask}`	클릭하면 실행할 **함수 자체**를 연결
`if (e.key === 'Enter')`	모든 키에서 실행되고, **Enter인지 검사**
`todos.filter((todo, index) => index !== indexToDelete)`	조건이 **true인 것만 남긴** 새 배열 → 삭제 효과
`todos.map((todo, index) => <li>...)`	배열의 각 요소를 `<li>`로 **변환**
`key={index}`	React가 목록 항목을 구분하는 이름표
`onDoubleClick={() => handleDeleteTask(index)}`	더블클릭 시 **자기 index**를 넣어 삭제 실행
`className="..."` ↔ `.클래스명 { }`	JSX 요소와 CSS 규칙 연결

30초 설명 템플릿

"TodoList는 함수형 컴포넌트이고, useState로 할 일 목록 `todos`(배열)와 입력값 `input`(문자열)을 기억한다. 입력창은 onChange로 입력값을 input에 반영하고 value로 그 값을 보여준다. 버튼 클릭이나 Enter가 handleAddTask를 실행하면, 공백 검사 후 `[...todos, input]`으로 새 배열을 만들어 setTodos하고 setInput('')으로 입력창을 비운다. 그러면 React가 다시 그리면서 todos.map으로 li를 만든다. li를 더블클릭하면 그 index를 뺀 새 배열을 filter로 만들어 setTodos해서 삭제한다."

15

복습 문제

답을 보기 전에 먼저 직접 써보자. 정답과 해설은 다음 장에 있다.

A. 개념 확인

Q1 개념

useState에서 첫 번째 값과 두 번째 값의 역할은?

Q2 개념

`todos`의 초기값 `[]`와 `input`의 초기값 `''`의 차이는? 각각 왜 그 모양인가?

Q3 개념

`setInput`은 어떤 역할을 하는가?

Q4 개념

`value={input}`은 어떤 역할을 하는가?

Q5 개념

`onChange`는 언제 실행되는가? milk를 입력하면 몇 번 실행되는가?

Q6 개념

`e.target.value`는 무엇인가?

Q7 개념

배열 `["milk", "egg", "apple"]`에서 첫 번째 요소의 index는? 마지막 요소의 index는?

Q8 개념

`filter()`는 어떤 요소를 남기는가?

B. 코드 결과 예측

Q9 결과 예측

다음 두 결과를 각각 쓰시오.

```
todos = ["milk", "egg"]
input = "banana"

[...todos, input]   // ①
[input]             // ②
```

Q10 결과 예측

`input`이 `"   "`(공백 3칸)일 때 Add Task를 클릭했다. 클릭 후 `todos`와 `input`은?

Q11 결과 예측

`input`이 `"  tea "`일 때 추가하면, todos에 저장되는 값은 정확히 무엇인가?

Q12 결과 예측

`todos = ["milk", "egg", "apple", "bread"]`일 때 `handleDeleteTask(2)`가 실행되면 새 todos는?

Q13 결과 예측

입력창에서 **A키**를 눌렀다. ① handleKeyDown은 실행되는가? ② handleAddTask는 실행되는가?

C. 빈칸 채우기

Q14 빈칸

빈칸 ㉠~㉤을 채우시오.

```
const [todos, ㉠______] = useState(㉡____);
const [input, setInput] = useState(㉢____);

onChange={(e) => setInput(e.㉣______.value)}

const newTodos = todos.㉤______((todo, index) => index !== indexToDelete);
```

D. 코드 흐름 순서 맞추기

Q15 순서

입력창에 egg를 적은 상태에서 **Enter**를 눌러 목록에 추가되기까지, 아래 보기를 순서대로 나열하시오.

ⓐ handleAddTask 실행 ⓑ onKeyDown 발생 ⓒ setTodos([...todos, input]) ⓓ e.key === 'Enter' 검사 ⓔ todos.map()으로 li 생성 ⓕ handleKeyDown 실행 ⓖ input.trim() !== '' 검사 ⓗ React 재렌더링 ⓘ setInput('')

Q16 순서

`todos = ["milk", "egg", "apple"]`에서 **apple을 더블클릭**해 삭제되기까지 순서대로 나열하시오.

ⓐ setTodos(newTodos) ⓑ handleDeleteTask(2) 실행 ⓒ onDoubleClick 발생 ⓓ 화면에서 apple 사라짐 ⓔ filter로 ["milk", "egg"] 생성 ⓕ React 재렌더링

E. 직접 설명하기

Q17 설명

`onClick={handleAddTask}`와 `onClick={handleAddTask()}`의 차이를 설명하시오.

Q18 설명

친구가 "Enter를 누르면 입력창이 비워지는 거지?"라고 말했다. 무엇이 부정확한지 코드 기준으로 고쳐 설명하시오.

Q19 설명

`map()`의 역할과, 콜백의 `todo`, `index`가 각각 무엇인지 설명하시오.

Q20 설명

`key={index}`는 왜 사용하는가?

Q21 설명

`onDoubleClick`에는 왜 `() => handleDeleteTask(index)`처럼 화살표 함수로 감싸서 넣었는가?

Q22 설명

`...todos`만으로 state가 바뀌지 않는 이유와, 실제로 state를 바꾸는 코드는?

정답과 해설

Q1 **첫 번째 = 현재 상태 값, 두 번째 = 그 상태를 변경하는 함수**
`[todos, setTodos]`에서 todos는 읽는 용도, setTodos는 바꾸는 용도다. 4장

Q2 **`[]`는 빈 배열, `''`는 빈 문자열**
todos는 할 일 **여러 개**를 담아 배열, input은 입력 중인 **글자 한 줄**이라 문자열. 헷갈림 주의 1

Q3 **입력창의 값을 input state에 반영(변경)한다**
onChange 안에서 `setInput(e.target.value)`로 호출된다.

Q4 **현재 input state 값을 입력창에 표시한다**
저장이 아니라 **표시**. 저장은 onChange → setInput. 헷갈림 주의 3

Q5 **입력창 내용이 바뀔 때마다 실행 · milk는 4번**
"m", "mi", "mil", "milk" 각각에서 실행된다.

Q6 **이벤트가 일어난 요소(input)에 지금 적혀 있는 값**
`e` = 이벤트 객체, `e.target` = 이벤트가 일어난 요소.

Q7 **첫 번째 0 · 마지막 2**
마지막 index = 개수(3) − 1. 헷갈림 주의 7

Q8 **조건 함수가 true를 돌려준 요소만 남긴다**
결과로 **새 배열**을 만든다. 삭제는 그 효과. 헷갈림 주의 6

Q9 ① `["milk", "egg", "banana"]` ② `["banana"]`
②를 setTodos에 넣으면 milk, egg가 사라진다. 6장

Q10 **todos는 그대로, input도 `"  "` 그대로**
`"  ".trim()`은 `''`라서 if 안쪽 전체(setTodos, setInput('')) 가 실행되지 않는다. **입력창도 비워지지 않는다**는 점이 함정.

Q11 **`" tea "` (앞뒤 공백 포함)**
L10은 검사할 때만 trim을 쓰고, L11은 원래 `input`을 넣는다.

Q12 `["milk", "egg", "bread"]`
index 2는 apple. `index !== 2`가 false인 apple만 빠진다.

Q13 **① 실행된다 ② 실행되지 않는다**
모든 키에서 handleKeyDown이 실행되고, `e.key`가 `'a'`라서 Enter 검사가 false. 헷갈림 주의 5

Q14 ㉠ `setTodos` ㉡ `[]` ㉢ `''` ㉣ `target` ㉤ `filter`

Q15 ⓑ → ⓕ → ⓓ → ⓐ → ⓖ → ⓒ → ⓘ → ⓗ → ⓔ
Enter → onKeyDown → handleKeyDown → Enter 검사 → handleAddTask → 공백 검사 → setTodos → setInput('') → 재렌더링 → map. setInput('')은 재렌더링 **전에** handleAddTask 안에서 호출된다(12장 POINT).

Q16 ⓒ → ⓑ → ⓔ → ⓐ → ⓕ → ⓓ
더블클릭 → handleDeleteTask(2) → filter로 새 배열 → setTodos → 재렌더링(map이 li 2개만 생성) → apple이 화면에서 사라짐.

Q17 **앞은 함수 자체를 넘겨 "클릭할 때" 실행, 뒤는 화면을 그리는 순간 바로 실행**

괄호 `()`는 "지금 실행"이라는 뜻이다. 뒤의 방식은 실행 결과(아무것도 없음)가 onClick에 들어가서, 클릭해도 원하는 동작이 일어나지 않는다.

Q18 **Enter는 handleAddTask를 실행시킬 뿐이고, 입력창을 비우는 건 그 안의 setInput('')이다**

모범 답안: "Enter를 누르면 handleKeyDown이 Enter인지 확인해서 handleAddTask를 실행하고, 공백 검사를 통과하면 setTodos 다음 줄의 setInput('') 때문에 input이 빈 문자열이 된다. 그래서 공백만 입력한 경우엔 Enter를 눌러도 비워지지 않는다." 헷갈림 주의 4

Q19 **map은 배열의 각 요소를 같은 방식으로 변환해 새 배열을 만든다**

이 코드에서는 문자열 하나하나를 `<li>`로 바꾼다. `todo` = 현재 처리 중인 실제 값(예: "egg"), `index` = 그 값의 배열 위치(예: 1).

Q20 **React가 목록의 각 항목을 구분하기 위한 이름표**

state가 바뀌어 다시 그릴 때, React는 key로 어떤 li가 어떤 li인지 알아본다. map으로 만든 목록에는 key를 붙인다.

Q21 **삭제할 index를 함께 넘기면서도, 더블클릭할 때 실행되게 하려고**

`handleDeleteTask(index)`를 그대로 쓰면 괄호 때문에 그리는 순간 실행된다. 화살표 함수로 감싸면 "더블클릭하면 이걸 실행해줘"라는 함수가 되고, 각 li는 자기 index를 품은 함수를 갖게 된다.

Q22 **...todos는 새 배열을 만드는 재료일 뿐, 실제 변경은 setTodos**

`[...todos, input]`은 새 배열 값을 만들 뿐이다. 그 배열을 `setTodos(...)`에 넣어야 state가 바뀌고 React가 다시 그린다. 헷갈림 주의 2

TIP **틀린 문제 활용법**

틀린 문제 옆에 적힌 장 번호로 돌아가 해당 코드 줄을 다시 읽고, **정답을 보지 않은 상태로 해설을 내 말로 다시 써보는 것**이 가장 효과적이다. 특히 Q10, Q11, Q15처럼 "대충 읽으면 틀리는" 문제는 코드를 한 줄씩 짚어보자.